Volund: An Empirical Study of Unattended Multiagent Program Optimization

Gustavo Wagon Widman
Instituto de Tecnologia e Liderança (Inteli)
São Paulo, Brazil

Laiza Ribeiro Silva
Instituto de Tecnologia e Liderança (Inteli)
São Paulo, Brazil

***Abstract*—We study an unattended multiagent procedure for optimizing one synthetic C++20 event range-statistics engine. Three separately initialized campaigns start from the same untuned indexed seed. A lead delegates repeated measurement and implementation work, while an independent reviewer reruns material evidence before endorsing hypotheses or source promotions. A frozen controller preserves exact artifacts, serializes native work, selects the first complete valid comparison, and requires reviewed task-based closure without a research time or token quota. All three campaigns completed, and their sealed endpoints passed the declared final validation. Fresh thirty-block comparisons gave transfer portfolio speedups of 2.240, 4.547, and 5.445; shifted-input speedups were 2.236, 5.186, and 5.294. One shifted family regressed: C1 sparse queries incurred approximately 9.7% higher latency. Thirteen hypotheses closed as nine supported, three contradicted, and one inconclusive, with eight source promotions. Observed campaign model tokens correspond to USD 17.08–18.23 under a dated Standard API-equivalent scenario; unreported compaction work and other coverage gaps prevent a whole-run cost bound.**

***Index Terms*—coding agents, program optimization, multiagent systems, empirical review, reproducibility**

I. Introduction

We report three unattended optimization campaigns that transformed the same untuned C++20 range-statistics engine into validated faster implementations. Each campaign combined delegated measurement and implementation with source-bound empirical review, then sealed an endpoint before a separate final assessment. The resulting transfer portfolio speedups were 2.240, 4.547, and 5.445. The retained evidence also captures negative hypotheses, a failed implementation and repair, a shifted-input regression, and incomplete resource-accounting coverage.

The contribution is an artifact-linked account of both optimization outcomes and the investigations that produced them. It connects exact candidate sources to correctness checks, paired timing measurements, hypothesis dispositions, review decisions, and direct seed-relative development trajectories. A frozen controller specifies how evidence authorizes promotion, how invalid timing can be retried without selecting a favorable score, and how a campaign reaches reviewed task-based closure. Reporting these rules alongside their observed execution makes the boundary between a proposed improvement and an accepted result inspectable.

Language-model program optimization already has direct precedents. PIE studies performance-improving C++ edits; AlphaEvolve uses program search with evaluator feedback; POLO and PerfAgent combine performance diagnosis with iterative refinement [1], [2], [3], [4]. The present study examines a specified end-to-end research workflow. A lead selects investigations and maintains campaign context; workers repeatedly measure, implement, test, and repair; a separate reviewer reruns material evidence before endorsing a claim or source promotion. A factual journal and automated controller retain evidence and corrections across these interactions.

The study asks three questions. **RQ1:** what valid seed-to-endpoint performance is observed on the prespecified transfer and shifted-input portfolios? **RQ2:** how do direct seed-relative development performance, hypotheses, repairs, nominations, and empirical reviews evolve in each campaign? **RQ3:** what elapsed time, observed model usage, native resources, and accounting gaps accompany the work? A campaign is the unit of replication; workload families, workers, hypotheses, and timing blocks are nested observations. No-gain, failed, stopped, and incomplete campaigns remain eligible outcomes under the protocol.

II. Related Work

Program optimization and iterative feedback. PIE separates functional conformance from runtime improvement and discusses variability in commodity-hardware measurements [1]. AlphaEvolve describes evolutionary program changes with automated feedback in a white paper [2]. POLO profiles project-level bottlenecks and uses collaborating agents for iterative rewrites [3]. PerfAgent, a preprint, supplies profiler and verifier feedback across repeated repository-level refinements [4]. These studies examine related mechanisms under different tasks and evaluation regimes.

Multiagent orchestration, review, and verification. AutoGen provides configurable agents, tool use, and programmable conversation patterns [5]. MetaGPT organizes software-development roles through standard operating procedures and incorporates execution-based feedback [6]. ChatDev distinguishes reviewer–programmer code review from tester–programmer dynamic testing [7]. AgentCoder separates programming, test design, and test execution, while documenting that generated test oracles can themselves be wrong [8]. Building on these role-specialization and executable-feedback approaches, an empirical review record binds a particular source and claim to the reviewer's own builds, correctness checks, profiles, and paired timings. We examine the resulting campaign evidence.

Evaluation scope. SWE-Perf and GSO evaluate optimization in real repositories [9], [10]. Our constructed engine per-

mits tighter control of its interface, invocation lifecycle, and source lineage, at the cost of much narrower external validity. Compiler-feedback experiments further motivate comparing iterative feedback with additional sampling [11].

Measurement and selection. Benchmarking research motivates repetition at appropriate levels and attention to incidental experimental choices [12], [13]. We retain paired observations and distinguish timing uncertainty from variation across campaigns. Selection-bias research motivates a separate final assessment [14].

III. Method

A. Study design and task

We froze the task, role instructions, controller, execution interfaces, measurement rules, and final assessment before three serial v3 campaigns. Each began with fresh participant contexts and a workspace containing the same original seed. Participants received the public task and their own campaign evidence; other campaigns and hidden assessment inputs and outcomes were outside their intended read scope. The design team had seen preliminary implementations and outcomes before this freeze. That work is excluded from the reported cohort, but fresh participant contexts do not remove this researcher-level information advantage. The supplement records exact artifacts and bindings.

The C++20 engine accepts events and queries through a fixed C ABI. Each query selects one stream or all streams within a half-open timestamp interval and returns exact count, signed sum, minimum, and maximum. Duplicate events count separately, input order is irrelevant, and empty results contain four zeros. The contract permits at most 1,000,000 events and 4,096 queries and specifies invalid-input handling, integer bounds, input immutability, and reentrancy. Persistent input-dependent indexes or answers are prohibited.

The untuned indexed seed builds timestamp-sorted global and per-stream indexes, finds interval bounds by binary search, and scans matching values. Index construction, query processing, and destruction are included in each measured invocation. Workers may change algorithms, layouts, allocation, batching, and parallel work within the portable standard-library contract. Canonical builds use C++20 at `-O3` with a generic x86-64 target; supplemental diagnostics cannot replace the official checks. An unchanged-program A/A comparison checks instrumentation. The supplement gives the exact ABI, compiler treatment, and preflight controls.

B. Agent workflow and review

The lead and reviewer request `gpt-6.1-sol` with `high` reasoning effort; execution workers request `gpt-6-luna` with `xhigh` effort. Provider fallback is disabled. Requested identifiers, accepted effort settings, and reported routing are recorded, although immutable provider model versions are unavailable. Harness design and validation are setup work, separate from the measured roles.

The lead delegates baseline measurement and profiling before making a measurement-grounded bottleneck claim. Workers repeatedly implement, measure, test, and repair candidates, returning evidence tied to exact sources. The lead may reuse workers, compare alternatives, merge compatible changes, or abandon branches. Parallel edits use separate branches with explicit merge ownership.

Before its discriminating test, each hypothesis records the baseline hash, motivating evidence, proposed mechanism, predicted and disconfirming observations, test conditions, and correctness/resource risks. Exploratory observations are labeled, and revisions append their rationale. Hypotheses close as supported, contradicted, or inconclusive; implementation defects are recorded separately. No hypothesis or improvement quota is imposed.

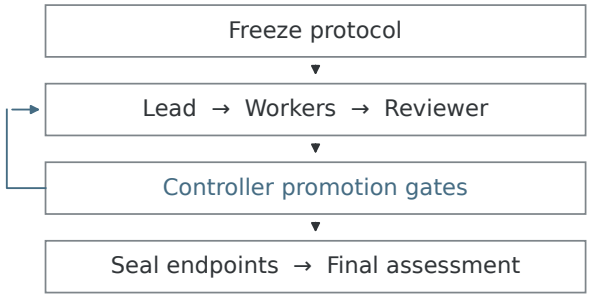


Fig. 1. Research and empirical review around a verified incumbent. Delegation, tests, repairs, and claim revision can repeat. Final assessment follows all endpoint seals.

The reviewer starts separately from proposing-worker conversations and checks completed hypotheses and material checkpoints, including negative findings. For promotion, it inspects the source and diff, rebuilds under official settings, reruns correctness and resource checks, collects its own paired comparison against the incumbent, and checks relevant profiles or traces. Memcheck covers all six public diagnostic families, supplemented by candidate-specific probes where needed. A reviewer who contributes an implementation fix cannot approve the resulting source.

Nomination records an immutable source, parent, build treatment, report, and evidence manifest. Promotion requires independent review and all semantic, resource, integrity, and performance gates. An immutable review plan selects the first complete valid official comparison for that source/reference pair; an unfavorable valid result cannot be replaced. Correctness and resource evidence must bind to the selected build. Confirmed semantic or definite Memcheck errors disqualify that source; repairs create new linked sources.

A controller serializes participant shell commands and native operations while allowing model planning, messages, and bounded structured reads. Timing recovery requires independently documented study-command overlap or identified external interruption, confirmed cleanup, and unchanged artifacts and procedures. The entire comparison repeats, without pooling partial samples. Valid poor scores and candidate failures do not qualify. Invalid attempts and queue delays remain in elapsed time and observed resources. Scheduling, retrieval, and recovery details appear in the supplement.

A factual journal retains source-bound findings, decisions, and appended corrections. Campaigns have no fixed research time or token budget. Completion requires a dossier resolving or bounding every hypothesis, review concern, failed branch, and actionable investigation, followed by reviewer-endorsed closure. Technical failure, cancellation, integrity stops, or session loss preserve incomplete status and the last verified incumbent. No human optimization steering is planned; interventions are disclosed separately.

C. Measurement and promotion

Six public workload families cover small build-dominated batches, sparse ranges, dense ranges, mixed global/per-stream queries, skew, and high stream cardinality. The evaluator fixes input lifecycle, warmup, calibration, resource limits, and invocation boundaries. Native execution uses the same two logical CPUs on a shared host. Callgrind diagnostics are separate from elapsed-time measurements [15].

Development comparisons use ten complete temporal blocks, each containing a reference/candidate pair for every family, with five pairs in each arm order per family. Both arms calibrate to a 0.05-second target, up to 65,536 repetitions, and use the larger count. Warmup and calibration are unscored; completed scored samples below target remain included and flagged.

For family w and block b, define $r_{w,b} = \ln\left(\frac{t_{R,w,b}}{t_{C,w,b}}\right)$ and $S_w = \exp\big(\mathrm{median}_b\, r_{w,b}\big)$. With $W = 6$, the equal-family portfolio is

$$G = \exp\left(\frac{1}{W}\sum_{w=1}^{W} \mathrm{median}_b\, r_{w,b}\right). \tag{1}$$

Values above one favor the candidate. Ten thousand bootstrap draws jointly resample complete blocks, reusing each draw's indices across families. The portfolio lower screen L is the interpolated 10th percentile, a nominal one-sided 90% percentile-bootstrap lower bound. Promotion requires $G \geq 1.03$, $L > 1$, every $S_w \geq \frac{1}{1.10}$, and all nonperformance gates. The family floor permits at most a 10% latency increase under this estimator. Incomplete comparisons supply no estimate. Exact schedules, seeds, interpolation, and technical limits are in the supplement.

D. Final assessment and trajectories

The endpoint is the last promoted source at endorsed closure, or the seed if none was promoted. Stopped campaigns retain incomplete status and their last verified incumbent, or an unavailable endpoint. All three endpoints must be sealed before final assessment or release of held-out results. The evaluator checks correctness and resource compliance in both final strata and reruns the six-family public Memcheck suite. A failed endpoint is not replaced with earlier code.

Eligible changed endpoints receive fresh paired seed comparisons on a primary same-distribution/new-seed transfer portfolio and a separate secondary portfolio with changed sizes/distributions. Each contains six families and thirty complete blocks, fifteen pairs in each arm order, using the same estimator and bootstrap. Final assessment has no promotion threshold and cannot authorize optimization, reselection, or replacement campaigns. The same-artifact first-valid recovery policy applies. Unchanged seeds are labeled algebraic identity; missing and invalid estimates remain visible.

After each promotion, the controller records a direct seed-versus-checkpoint comparison on public development inputs. It reuses the selected independent comparison when the parent is the seed; otherwise it collects an additional ten-block comparison. Trajectories use comparison completion time, retain promotion time separately, and never multiply parent-relative gains. The initial ratio of one denotes source identity. Usage/cost coordinates require matching authoritative coverage; later receipts are not interpolated.

E. Resource accounting

The public Codex 0.159.3 stream supplies cumulative thread counters. We reconcile one endpoint per unique thread, counting cached input and reasoning within input and output, respectively. Failed and interrupted work remains in scope, with unreported tails labeled unknown. Parser defaults leave cache-write provenance uncertain; remote-compaction usage can fall outside these counters [16], [17].

Dated model-specific Standard API-equivalent prices yield hypothetical observed-token cost scenarios, with cache-write and price-band uncertainty retained [18], [19]. These scenarios are not actual bills or bounds on unreported work. Campaign elapsed time, queue delay, native resources, setup, and final assessment are reported separately. The supplement provides clock definitions, reconciliation rules, pricing formulas, and retained-artifact details.

IV. Results

A. Completion and endpoint validity

All three campaigns reached reviewer-endorsed task completion, and the automatic final assessment completed in its first invocation. The final endpoint was the last promoted source in every campaign. All 21 role threads and 68 turns reached completed status; campaign summaries reported no runtime failure. No operator optimization steering or measured-protocol changes were reported; ordinary observer activity on the shared host is disclosed in the supplement. The final evaluator rebuilt the seed and all three endpoints, obtained 89 passing correctness cases per source in each stratum, and completed all six public Memcheck probes per source.

The six final performance comparisons all used their first complete valid attempt. No final endpoint was repaired or replaced after assessment. Supplementary provenance records bind the endpoint sources and validation coverage to the immutable report.

B. Final artifact performance

The primary transfer and separate shifted-input portfolios both favored every final endpoint (Table I; Fig. 2). Transfer speedups ranged from 2.240 to 5.445, and shifted-input speedups from 2.236 to 5.294.

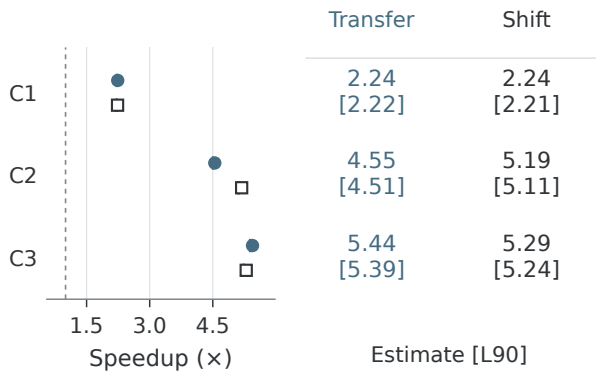


Fig. 2. Final seed-to-endpoint portfolios. Filled circles denote transfer; open squares denote shift. Points are equal-family effects and stems end at nominal one-sided 90% lower bounds (L90), from thirty paired blocks per stratum.

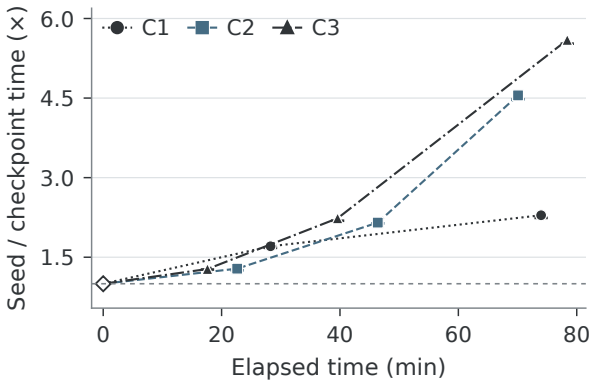


Fig. 3. Direct seed-relative development observations at comparison completion, measured from campaign-owner start. The hollow diamond at zero is unmeasured source identity. Lower stems are nominal one-sided 90% bounds for individual comparisons, not a simultaneous band.

TABLE I
FINAL SEED-TO-ENDPOINT PORTFOLIO RESULTS

Campaign	Stratum	Speedup	L90
C1	Transfer	2.240	2.222
C1	Shift	2.236	2.214
C2	Transfer	4.547	4.507
C2	Shift	5.186	5.114
C3	Transfer	5.445	5.390
C3	Shift	5.294	5.235

Each row uses thirty paired blocks and the first complete valid attempt. L90 is the nominal one-sided 90% complete-block bootstrap lower bound. Transfer and shift are separate six-family portfolios.

The portfolio gains were not uniform. C1′s shifted sparse-family speedup was 0.911, corresponding to approximately 9.7% higher latency. Its other shifted families ranged from 1.612 to 5.271. Every transfer family improved in all three campaigns, and every shifted family improved in C2 and C3. The largest family effects occurred on mixed global/per-stream queries: transfer speedups were 4.370, 9.885, and 14.607. Complete family effects and absolute per-invocation timings appear in the supplement.

C. Investigations and development trajectories

TABLE II
CAMPAIGN PROCESS OUTCOMES

Campaign outcome	C1	C2	C3
Role threads	7	8	6
Completed turns	20	21	27
Hypotheses (S / C / I)	3/2/0	3/0/1	3/1/0
Review records	8	7	7
Source promotions	2	3	3

S / C / I: supported / contradicted / inconclusive. Review records cover claims and nominations; three closure endorsements are separate.

All thirteen hypotheses had source-bound independent reviewer build, profile, and benchmark evidence. Nine closed as supported, three as contradicted, and one as inconclusive (Table II). Across the campaigns, 357 official operations included 46 benchmarks, all on first attempts. Eight nominations were promoted; none was left pending or rejected. Rejected exploratory approaches were distinct from formal nominations. The direct seed-relative development checkpoints were 1.706 and 2.291 for C1; 1.284, 2.153, and 4.551 for C2; and 1.277, 2.234, and 5.596 for C3 (Fig. 3). Each point uses its own seed comparison. The curves summarize selected development checkpoints, with no performance assigned between observations and no multiplication of parent-relative effects.

The retained investigations explain why performance and hypothesis disposition differ. In C1, a contiguous requested-index proposal lost performance on sparse and dense inputs. Selective 64-event summaries reduced repeated wide-range scanning, but initially increased high-cardinality overhead; sparse allocation of summary metadata addressed that cost. An unconditional flat scatter then regressed dense inputs and remained exploratory. The final promoted source selected between vector and flat builders using a deterministic sample of current-input stream membership. Its first implementation failed eleven correctness cases with segmentation faults because a 32-bit tag truncated a size-sized sentinel. A distinct repaired source used consistent widths and passed the required checks. The failed source remains in the record.

C2 and C3 investigated query-demand pruning, shared timestamp sorting, demand-amortized block summaries, and portable radix sorting of large arrays. Their initial pruning implementations retained family losses; shared sorting removed redundant work, and summaries addressed repeated range scans. Later sources used input-derived varying timestamp bytes to skip constant radix passes. Public diagnostic inputs can fall below the large-array radix threshold, so their instruction counts alone cannot explain the final radix-related timing gains. C3′s large-path instrumentation gap at closure was later supplemented by five clean fixed-endpoint Memcheck probes. These post-hoc checks do not change original acceptance, performance, or usage records.

D. Observed resources

Runtime-host wall intervals were 84.80, 74.25, and 86.75 minutes for C1–C3, including closing activity after endpoint

sealing. Owner-start to last observed usage spanned 84.93, 74.38, and 86.89 minutes. The supplement distinguishes these clocks from source-seal intervals. Final assessment added 14.47 minutes.

Observed unique-thread totals were 39.991, 44.941, and 43.954 million tokens. Input accounted for 128.066 million of the combined 128.886 million tokens; 123.561 million input tokens were reported cache reads. Repeated cached context contributes to these totals, so they are not counts of unique task content. Output already includes reasoning tokens.

A post-freeze response-band reconciliation assigned the observed cumulative increments to the short-input Standard pricing band, preserving unknown cache writes. It produced campaign subtotals of USD 5.491–5.904, 5.655–5.987, and 5.930–6.339, or USD 17.075–18.230 in total. The frozen broad scenarios remain in the supplement. C1 and C3 each showed compaction in an implementation context and a reviewer context, whose unreported usage is not recovered; C2 had no observed compaction marker. Provider request IDs, actual billing tier, and authoritative cache-write charges remain unavailable. Setup, harness authoring, interpretation, and manuscript preparation are outside this campaign subtotal. The interval neither measures an actual bill nor bounds total project cost.

V. Discussion and Conclusion

Across three campaigns, Volund produced endpoints with transfer portfolio speedups of 2.240, 4.547, and 5.445 over the indexed seed. All three endpoints also improved the shifted-input portfolio, although C1′s sparse queries incurred approximately 9.7% higher latency. Family-level results are therefore important: equal-weight portfolio gains can coexist with a workload-specific regression. Deployment benefit would depend on the actual request distribution and absolute costs.

The investigation records connect these outcomes to competing approaches, failed implementations, repairs, and empirical review. C1′s sentinel-width defect separated a promising optimization from a correct implementation; C2′s inconclusive pruning hypothesis separated an exploratory improvement from support for its proposed mechanism. C2 and C3 ultimately adopted radix-based approaches, while C1 combined conditional index construction with summaries. These records show how the campaigns developed and tested different solutions to the same task.

The study comprises three campaigns on one synthetic task with an untuned seed and no matched alternative workflow or research budget. It characterizes the complete procedure rather than estimating comparative advantage or individual component effects. The campaigns share task, host, service, and design choices. Development checkpoints are adaptively selected; separate final inputs reduce reuse of development evidence [14]. Bootstrap bounds assume exchangeable blocks and describe within-comparison timing uncertainty, not campaign-wide error control. Shared-host timing [13], finite correctness and memory checks, and potentially shared worker, reviewer, or test-oracle errors remain limitations. Observed-token estimates exclude unreported provider work and setup.

Volund provides an inspectable record of unattended optimization, from initial investigations to reviewed endpoints and separate final assessment. Evaluation on additional tasks, tuned baselines, and matched workflows would establish how these findings extend beyond the present case.

Data and Code Availability. Code, protocol, measurements, process records, and analysis scripts will be released upon publication at the Volund repository. Source hashes and reproduction instructions appear in the supplement.

Disclosure of AI Use. OpenAI generative AI tools assisted manuscript preparation and revision, supporting software, analyses, and visualizations. Their experimental use is described in Method. The authors take responsibility for the content.

References

[1] A. Shypula *et al.*, "Learning Performance-Improving Code Edits," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/ce65173b994cf7c925c71b482ee14a8d-Abstract-Conference.html

[2] A. Novikov *et al.*, "AlphaEvolve: A coding agent for scientific and algorithmic discovery." [Online]. Available: https://arxiv.org/abs/2506.13131v1

[3] J. Bai, R. Xu, S. Wu, D. Yang, J. Zhao, and G. Chen, "POLO: An LLM-Powered Project-Level Code Performance Optimization Framework," in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, International Joint Conferences on Artificial Intelligence Organization, 2025, pp. 7319–7328. doi: 10.24963/ijcai.2025/814.

[4] R. Deng *et al.*, "PerfAgent: Profiler-Guided Iterative Refinement for Repository-Level Code Optimization." [Online]. Available: https://arxiv.org/abs/2607.19653v1

[5] Q. Wu *et al.*, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations," in *First Conference on Language Modeling*, 2024. [Online]. Available: https://www.microsoft.com/en-us/research/wp-content/uploads/2023/08/LLM_agent.pdf

[6] S. Hong *et al.*, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/6507b115562bb0a305f1958ccc87355a-Abstract-Conference.html

[7] C. Qian *et al.*, "ChatDev: Communicative Agents for Software Development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2024, pp. 15174–15186. doi: 10.18653/v1/2024.acl-long.810.

[8] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "Agent-Coder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation." [Online]. Available: https://arxiv.org/abs/2312.13010

[9] X. He *et al.*, "SWE-Perf: Can Language Models Optimize Code Performance on Real-World Repositories?," in *Proceedings of the 43rd International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 306. PMLR, 2026, pp. 41651–41666. [Online]. Available: https://proceedings.mlr.press/v306/he26y.html

[10] M. Shetty, N. Jain, J. Liu, V. Kethanaboyina, K. Sen, and I. Stoica, "GSO: Challenging Software Optimization Tasks for Evaluating SWE-Agents," in *Advances in Neural Information Processing Systems*, 2025. doi: 10.52202/085713-5054.

[11] D. Grubisic, C. Cummins, V. Seeker, and H. Leather, "Compiler generated feedback for Large Language Models." [Online]. Available: https://arxiv.org/abs/2403.14714v1

[12] T. Kalibera and R. Jones, “Rigorous Benchmarking in Reasonable Time,” in *Proceedings of the 2013 International Symposium on Memory Management*, in ISMM '13. ACM, 2013, pp. 63–74. doi: 10.1145/2464157.2464160.

[13] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing Wrong Data Without Doing Anything Obviously Wrong!,” in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, in ASPLOS '09. ACM, 2009, pp. 265–276. doi: 10.1145/1508244.1508275.

[14] G. C. Cawley and N. L. C. Talbot, “On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation,” *Journal of Machine Learning Research*, vol. 11, no. 70, pp. 2079–2107, 2010, [Online]. Available: https://www.jmlr.org/papers/v11/cawley10a.html

[15] Valgrind Developers, “Callgrind: A Call-Graph Generating Cache and Branch Prediction Profiler.” [Online]. Available: https://valgrind.org/docs/manual/cl-manual.html

[16] OpenAI, “Codex Response Usage Parser.” [Online]. Available: https://github.com/openai/codex/blob/rust-v0.159.3/codex-rs/codex-api/src/sse/responses.rs#L138

[17] OpenAI, “Codex Remote Compaction and Usage Accounting.” [Online]. Available: https://github.com/openai/codex/blob/rust-v0.159.3/codex-rs/core/src/compact_remote_v2.rs#L223

[18] OpenAI, “OpenAI API Pricing.” 2026. [Online]. Available: https://developers.openai.com/api/docs/pricing

[19] OpenAI, “Prompt Caching.” 2026. [Online]. Available: https://developers.openai.com/api/docs/guides/prompt-caching